Computer System Architecture

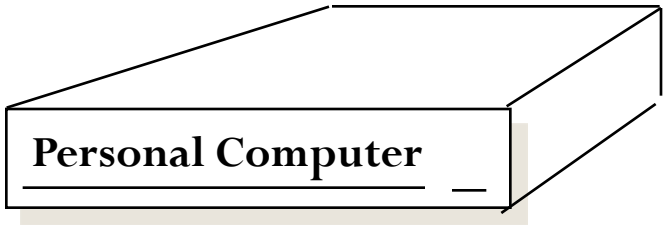
ISA (Instructions/address format)

Tradeoffs

Instruction Cycle, Datapath & Control (Microarchitecture)

Single Cycle vs. Multi Cycle

Where We Are: Comp Sys Architecture



Chap 2 : 2 weeks
Chap 8 (Stallings): 1 week
Chap 5 : 1 weeks
Chap 4 : 2 weeks

Instruction Set
Instructions format
Addressing modes

Chap 2 : ISA

Computer

Processor (CPU)

Control ("brain")

Datapath ("brawn")

Memory

Devices

Input

Output

Self Study

Arithmetic-Logic Unit (ALU)

Processor = Chaps. 4 NOW

Chap. 5 (done)

Chap. 8
Chap. 3.3, 3.4, 3.6
Stallings Book

ISA: General Purpose Registers

Advantages of registers:

1. registers are faster than memory

2. registers are easier for a compiler to use

e.g., (A*B) – (C*D) – (E*F) can do multiplies in any order

3. registers can hold variables

memory traffic is reduced, so program is sped up

(since registers are faster than memory)

code density improves (since register named with fewer bits

than memory location)

MIPS Registers:

32 x 32-bit GPRs, 32 x 32-bit FP regs

Top 10 Instructions are Simple: Reg mode

Rank	instruction	Average Percent total executed
1	load	22%
2	conditional branch	20%
3	compare	16%
4	store	12%
5	add	8%
6	and	6%
7	sub	5%
8	move register-register	4%
9	call	1%
10	return	1%
	Total	96%

- Simple instructions dominate instruction frequency

Basic ISA Classes

Accumulator (1 register):

1 address	add A	acc ← acc + mem[A]
1+x address	addx A	acc ← acc + mem[A + x]

Stack:

0 address	add	tos ← tos + next ; tos: Top of Stack

General Purpose Register:

2 address	add A B	EA(A) ← EA(A) + EA(B)
3 address	add A B C	EA(A) ← EA(B) + EA(C)

Load/Store (Reg-Mem):

load Ra Rb	Ra ← mem[Rb]
store Ra Rb	mem[Rb] ← Ra

Memory to Memory:

All operands and destinations can be memory addresses.

Comparing ISA Performance

Comparison:
Bytes per instruction?
Cycles per instruction?
Number of Instructions?

° **Code sequence for C = A + B for four classes of instruction sets:**

Stack	Accumulator	Register (1-register)	Register (3-register)
Pop A	Load A	Load R1,A	Load R1,A
Pop B	Add B	Add R1,B	Load R2,B
Add	Store C	Store C, R1	Add R3,R1,R2
Push C			Store C,R3

RISC machines (like MIPS) have only load-store instns via registers. So, they are also called load-store machines. CISC machines may even have memory-memory instrns, like mem (A) = mem (B) + mem (C) Advantages/disadvantages?

Addressing Modes (MIPS +Other ISA)

Addressing mode	Example	Meaning
Register	Add R4,R3	R4← R4+R3
Immediate	Add R4,#3	R4 ← R4+3
Displacement /Offset	Add R4,100(R1)	R4 ← R4+Mem[100+R1]
Register indirect	Add R4,(R1)	R4 ← R4+Mem[R1]
Indexed / Base	Add R3,(R1+R2)	R3 ← R3+Mem[R1+R2]
Direct or absolute	Add R1,(1001)	R1 ← R1+Mem[1001]
Memory indirect	Add R1,@(R3)	R1 ← R1+Mem[Mem[R3]]

Addressing Mode Usage: (ignore register mode)

3 programs measured on machine (VAX) with all memory address modes

---	**Displacement:**	**42% avg, (32% to 55%)**
---	**Immediate:**	**33% avg, (17% to 43%)**
---	**Register deferred (indirect):**	**13% avg, (3% to 24%)**
---	**Memory indirect:**	**3% avg, (1% to 6%)**
---	**Misc:**	**2% avg, (0% to 3%)**

75% displacement & immediate
88% displacement, immediate & register indirect

Immediate Size:

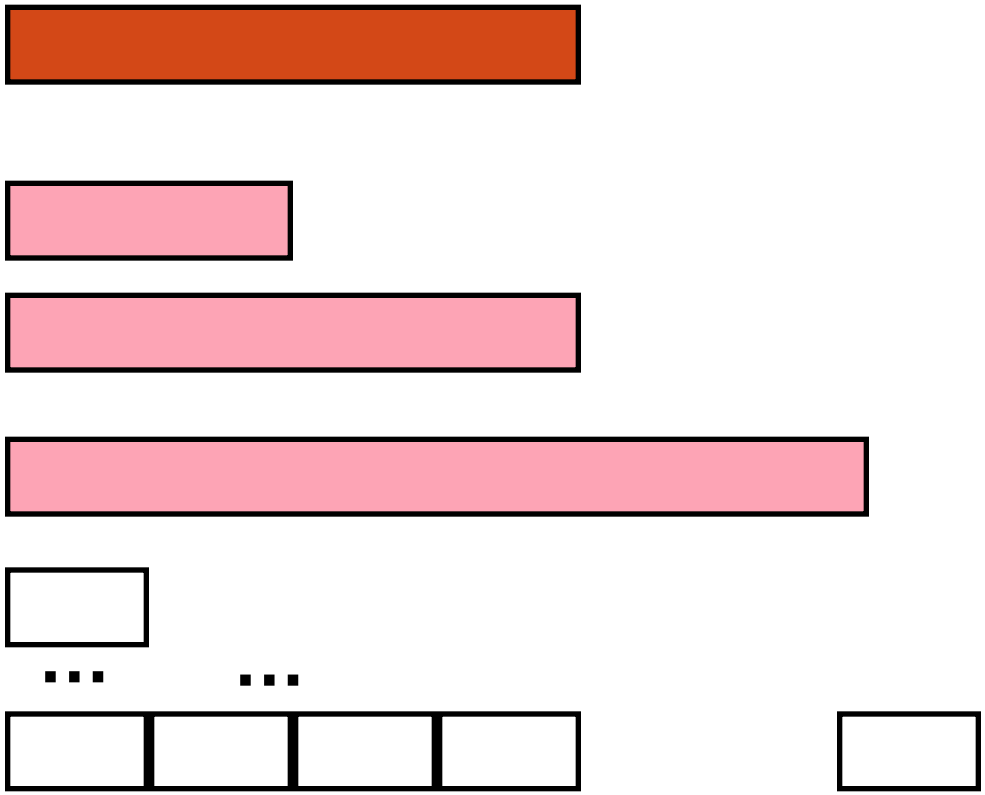
50% to 60% fit within 8 bits

75% to 80% fit within 16 bits

Instruction Formats: Bytes per Instr

Fixed

Variable:



Instruction Formats: Performance/Code size

- **If code size is most important, use variable length instructions:**

  (1)Difficult control design to compute next address

  (2) complex operations, so use microprogramming

  (3) Slow due to several memory accesses

- **If performance is over is most important, use fixed length instructions**

  (1) Simple to decode, so use hardware

  (2) Wastes code space because of simple operations

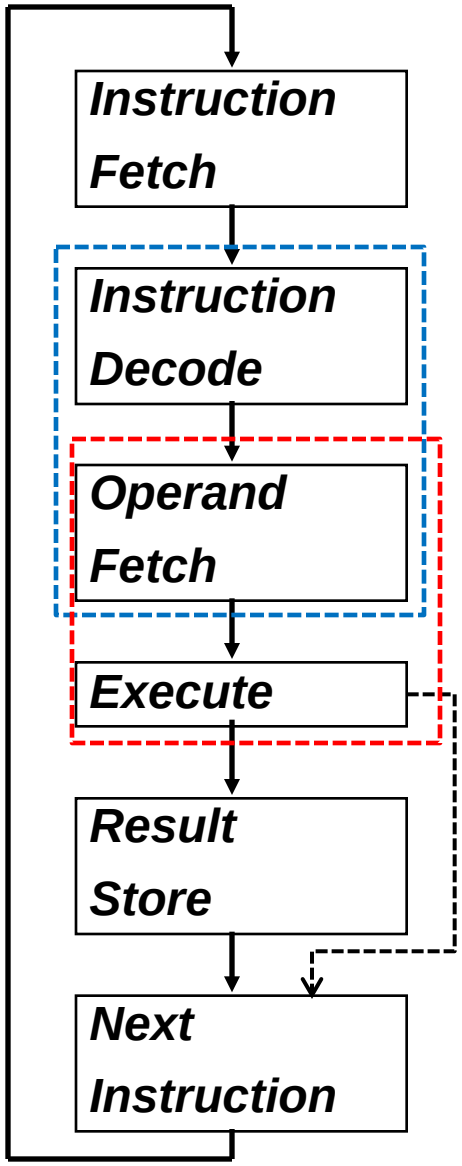
  (3) Works well with pipelining

- **ARM, MIPS added optional mode to execute subset of 16-bit wide instructions (Thumb, MIPS16); per procedure decide performance or density**

MIPS-lite processor

- Subset of MIPS instruction set ("MIPS-lite")
  - just enough to illustrate key ideas
  - instruction set subset (3 groups):
    - arithmetic-logical: `add`, `sub`, `and`, `or`, `slt`
    - memory reference: `lw`, `sw`
    - control flow: `j`, `beq`
    - *can we write real programs with just these?*
- Need up to 5 steps to execute any instruction in our subset:

Instruction Cycle



Obtain instruction from program storage

Determine required micro-actions for instruction

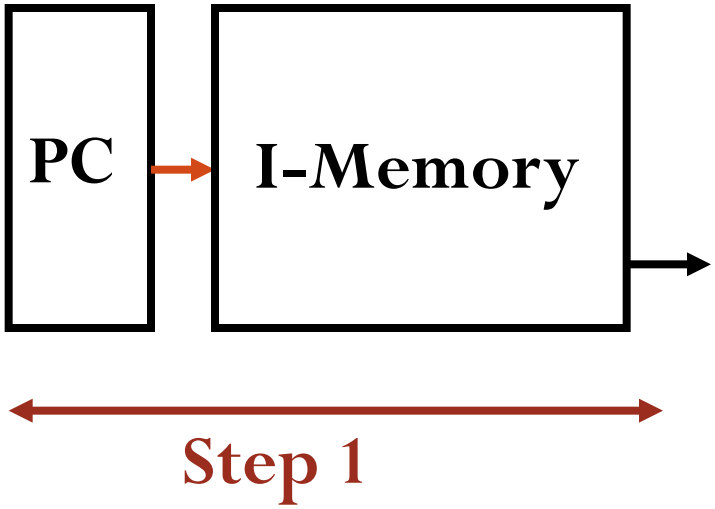
Locate and obtain operand data

Compute result value or status

Deposit results in storage for later use

Determine successor instruction/ Branch address

Building a Datapath for MIPS (step 1)



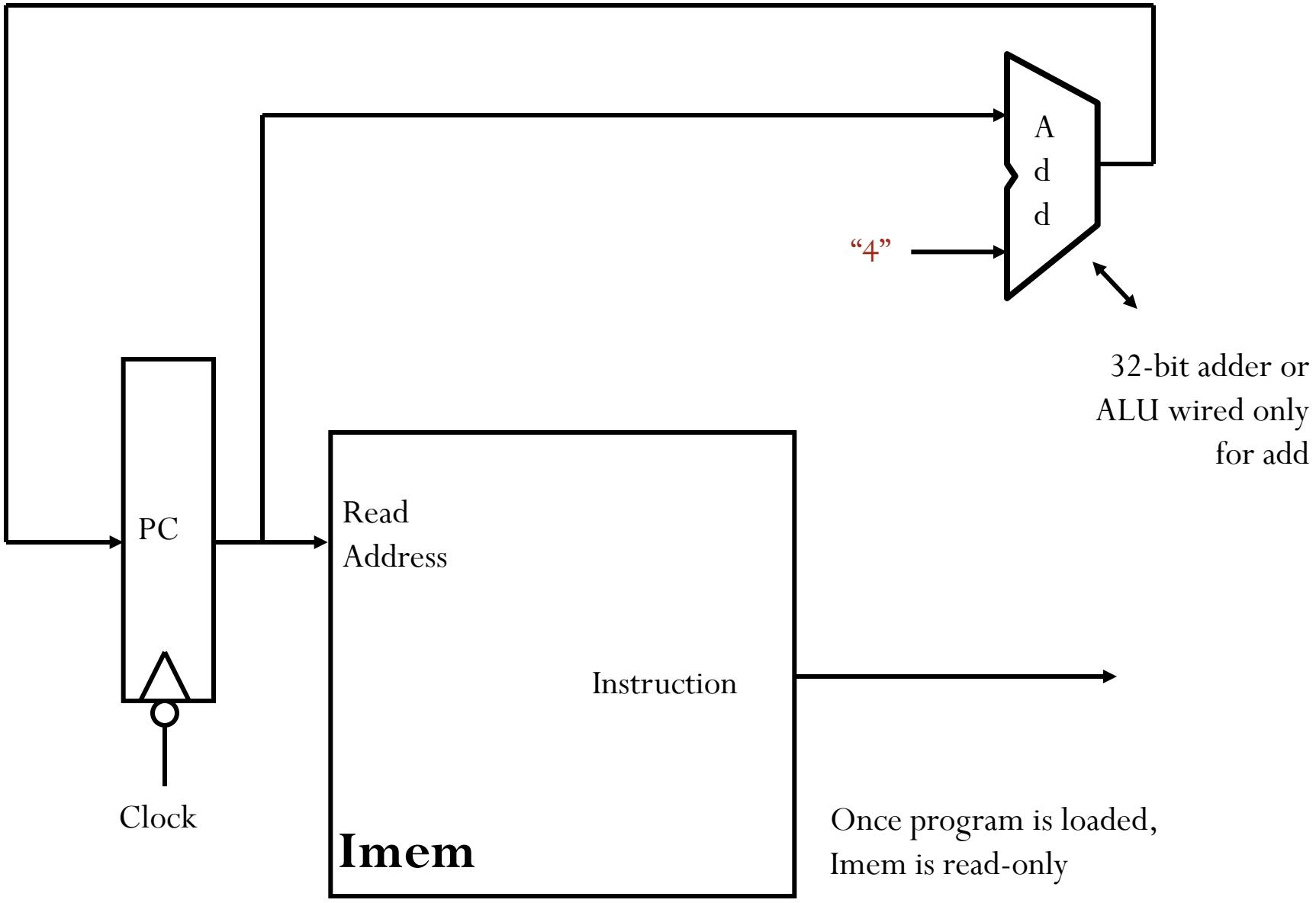
°Need up to 5 steps to execute any instruction:

- Step 1: fetch instruction

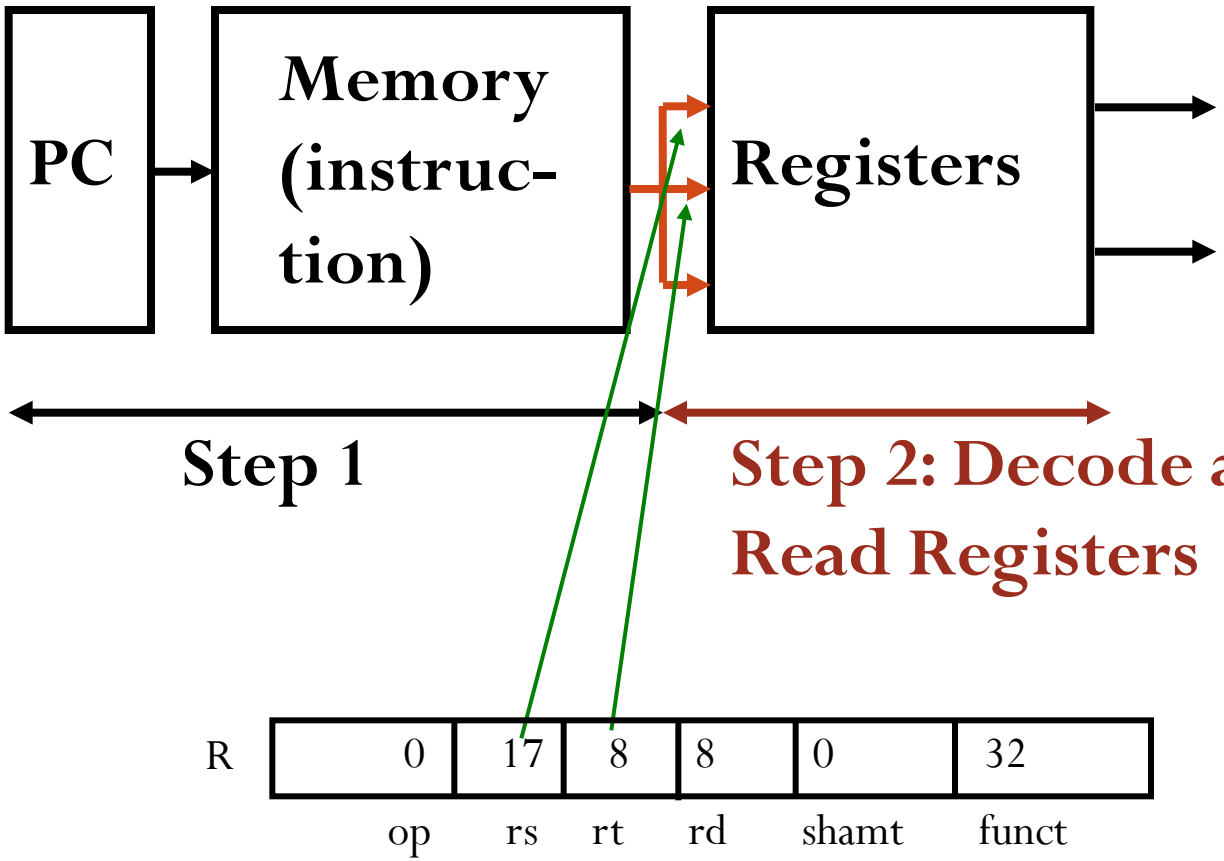
```
.          .
.          .
.          .
PC-4       add $t0,$t0,$t0
PC         add $t0,$s1,$t0
PC+4       lw $t1,20($s0)
PC+8       sw $t1,4($t0)
.          .
.          .
```

Flow of execution

Datapath Step 1: Instruction Fetch

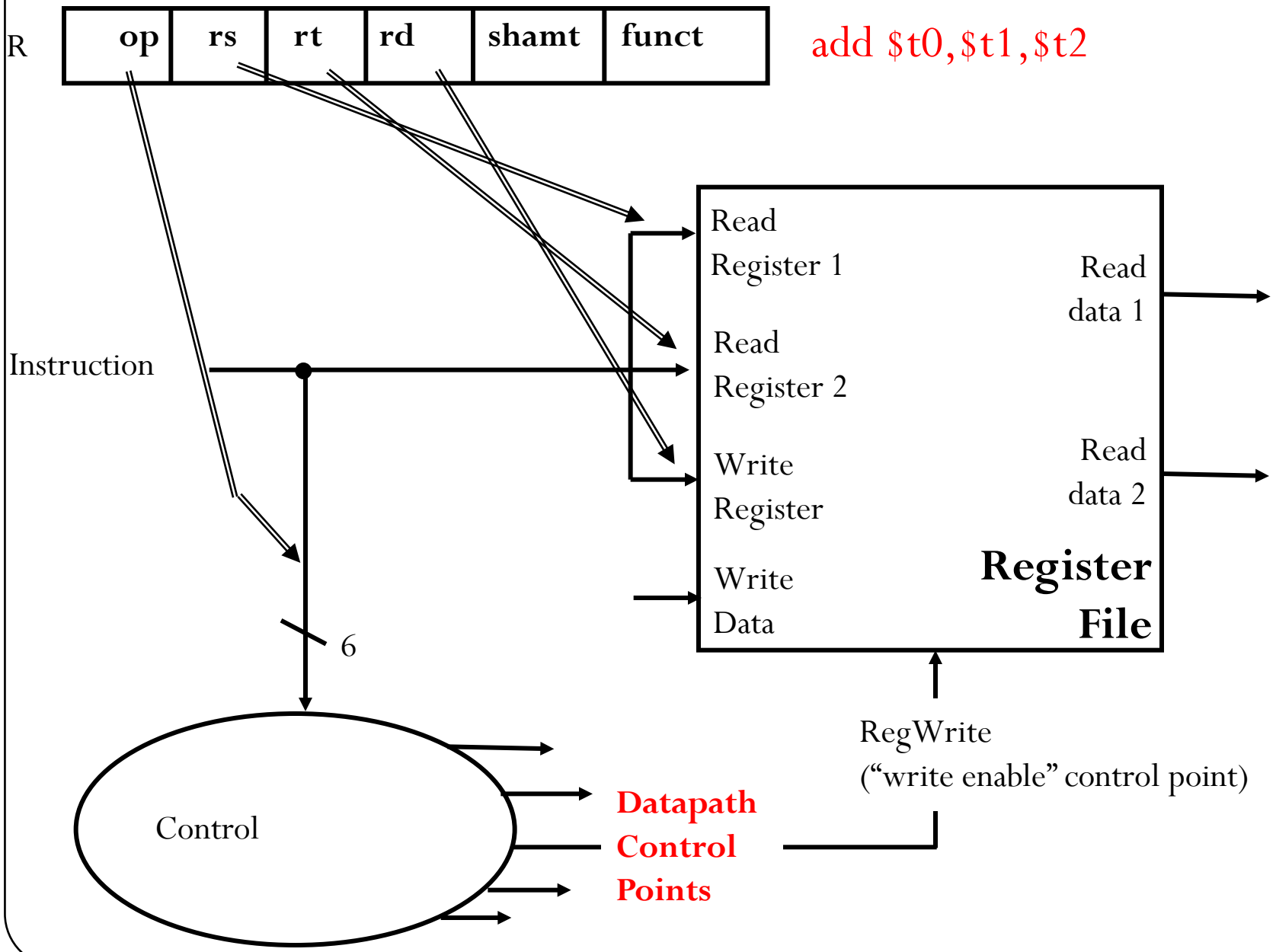


Building a Datapath for MIPS (step 2)



add $t0,$s1,$t0

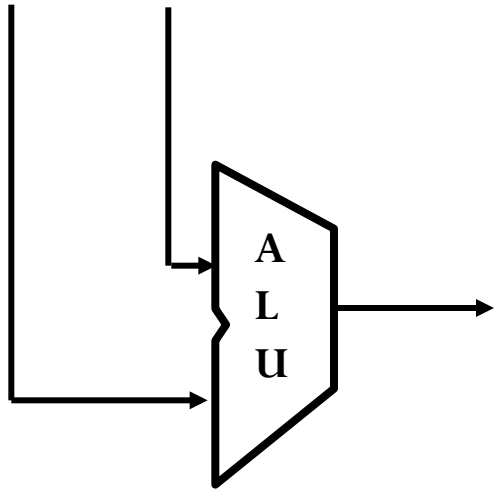
Datapath Step 2: Instruction decode



≤ 5 Steps in Executing MIPS Subset

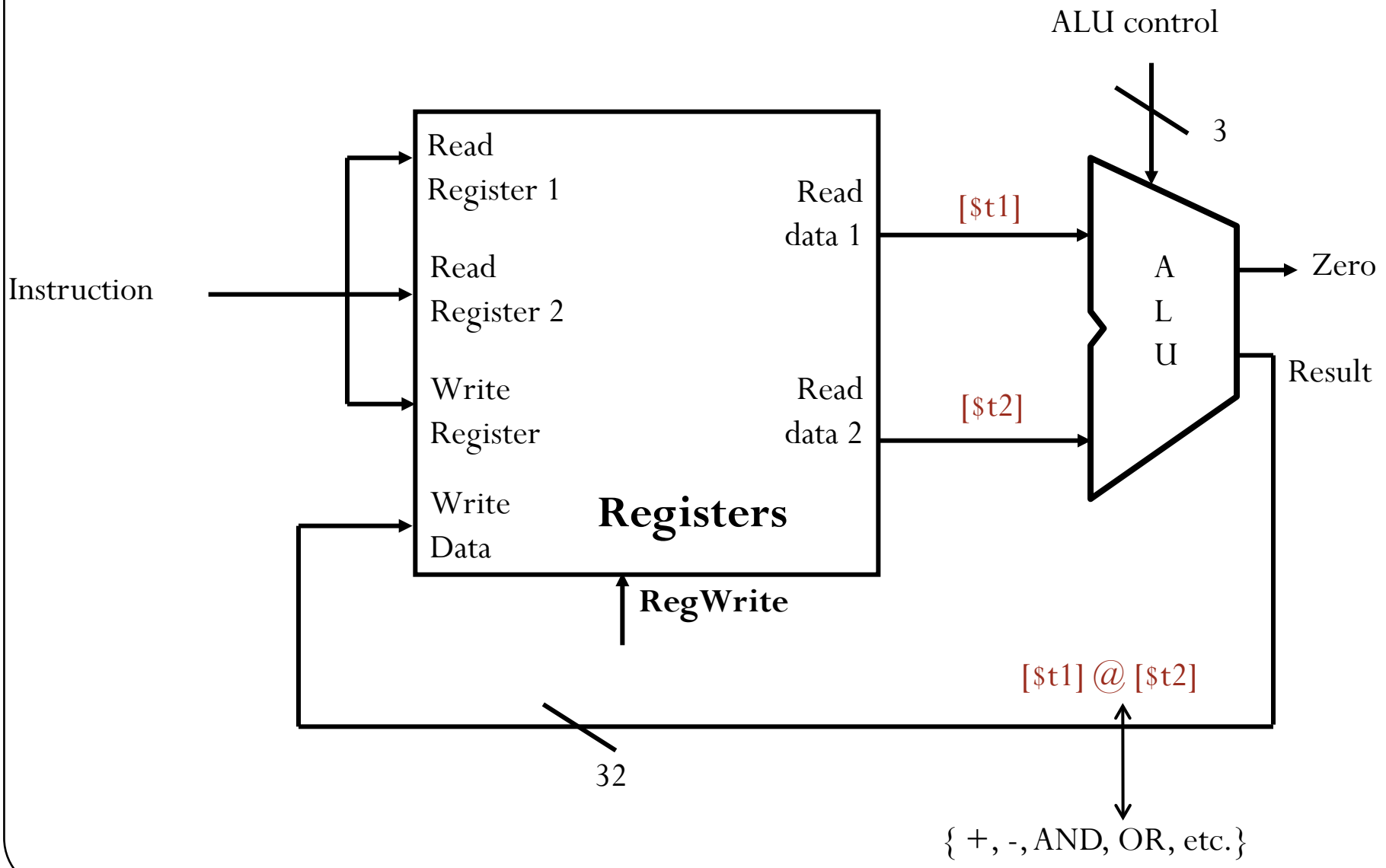
- 3rd step (Exec) onwards depends on instruction class
- EX: for ALU instructions, add $t0, $t1, $t2 outputs from registers t1 and t2 will be sent to the ALU input.
- For Memory-reference instruction: A*ddress* ⇐ *Base* + *offset*

```
lw $t0,20($s0)
```



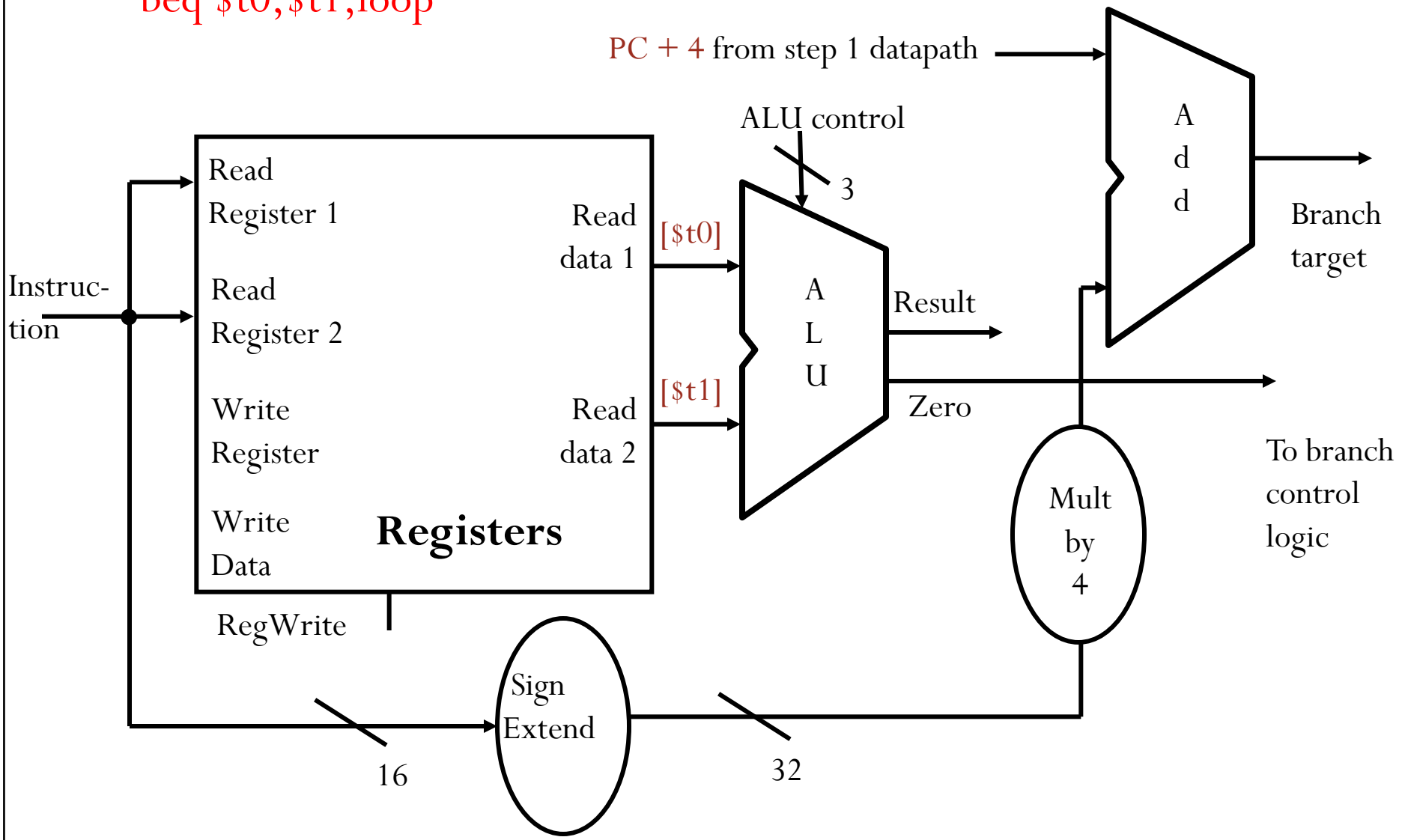
Datapath Step 3-4: R-format Instructions

add, sub, and, or, slt

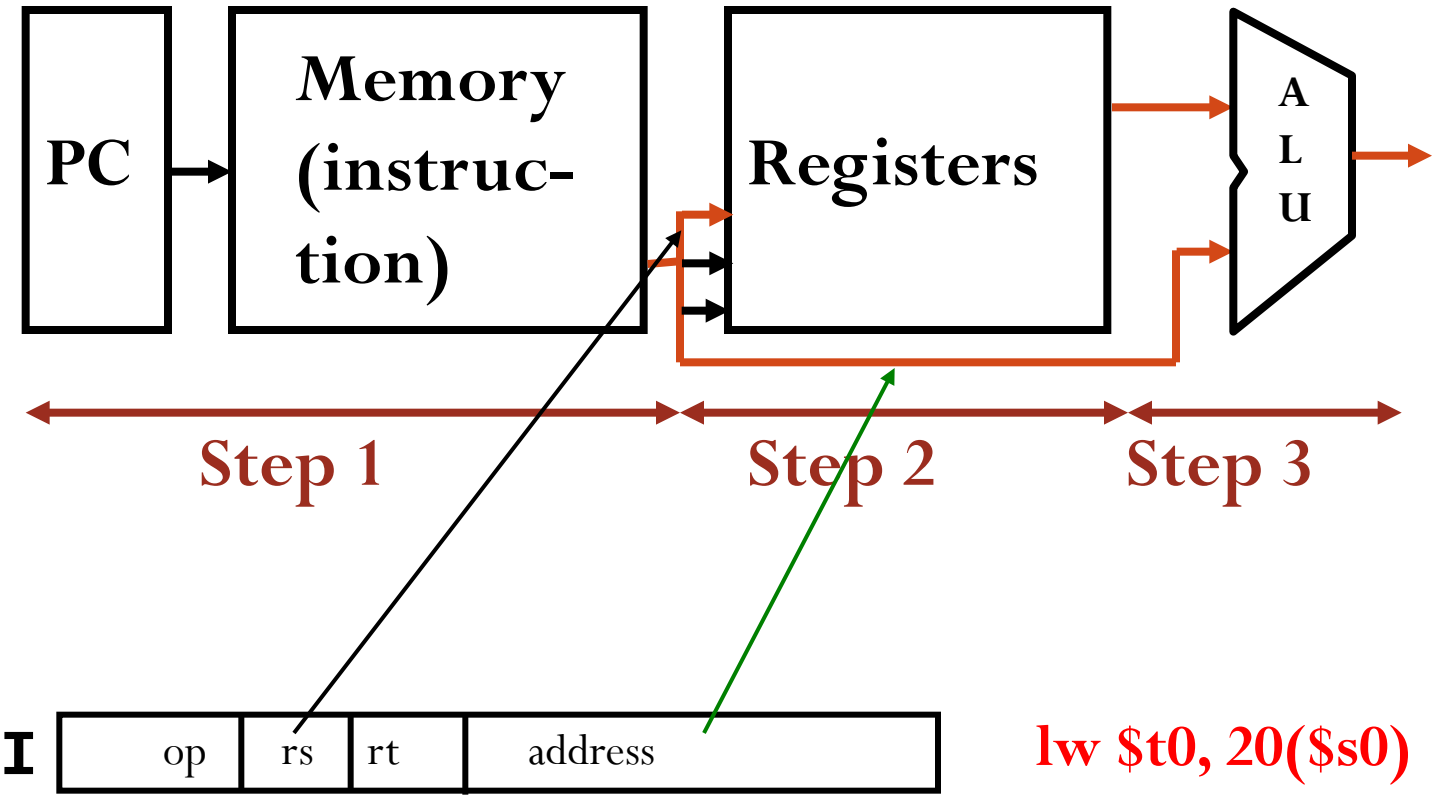


Datapath Step 3: Branch Instr

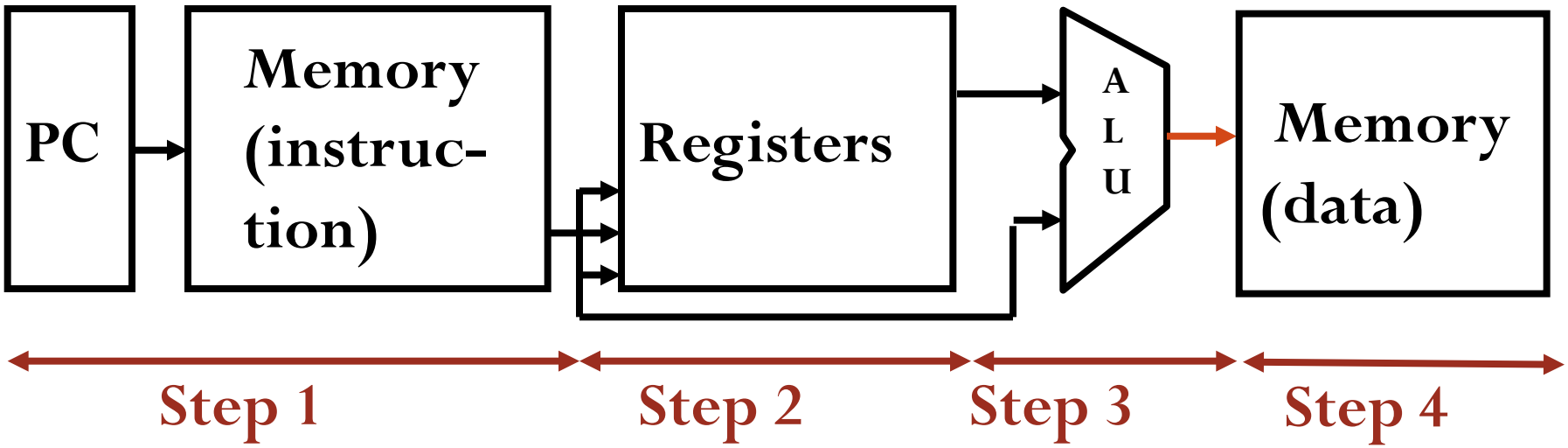
beq $t0,$t1,loop



Building a Datapath for MIPS (lw step 3)



Building a Datapath for MIPS (lw step 4)




```
.
.
.
add $t0,$t0,$t0
add $t0,$s1,$t0
lw $t1,20($s0)
sw $t1,4($t0)
.
.
```

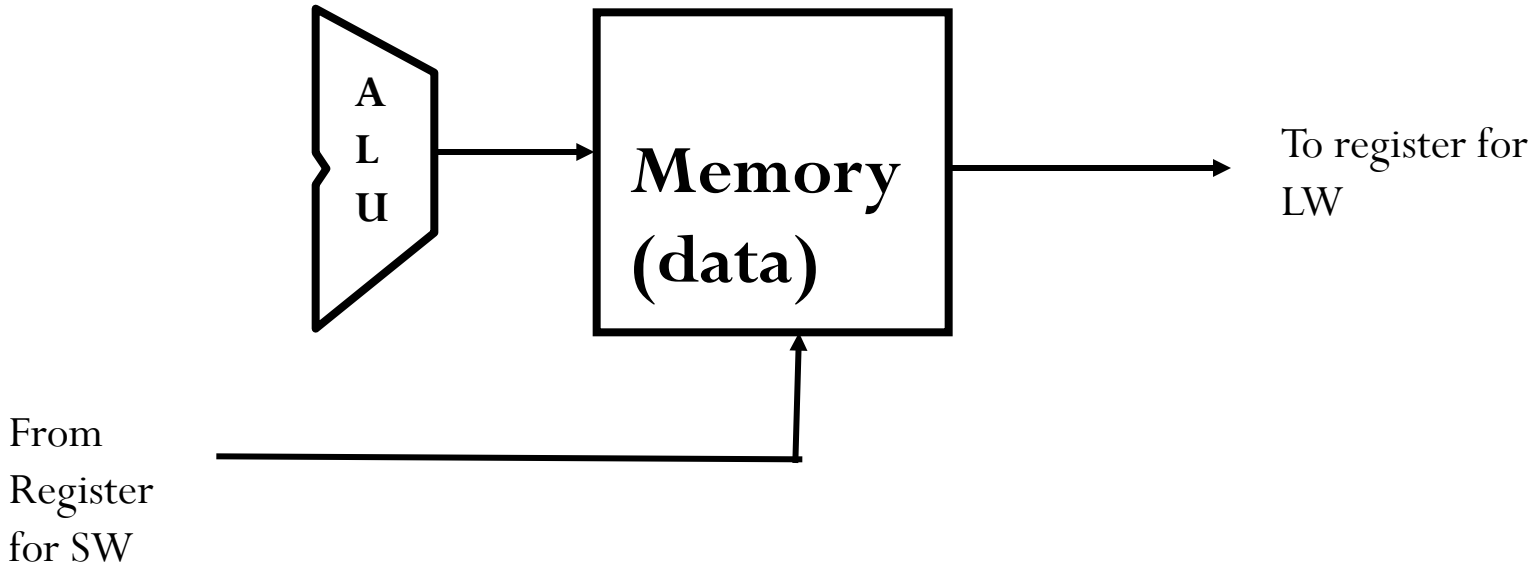
≤ 5 Steps in Executing MIPS Subset

- 4th step: Data memory access (offset addressing mode)

Ex: for `lw`: Fetch Data from Memory

Data ⇐ *Mem[Address]*

For sw: Put the contents of a register in Memory

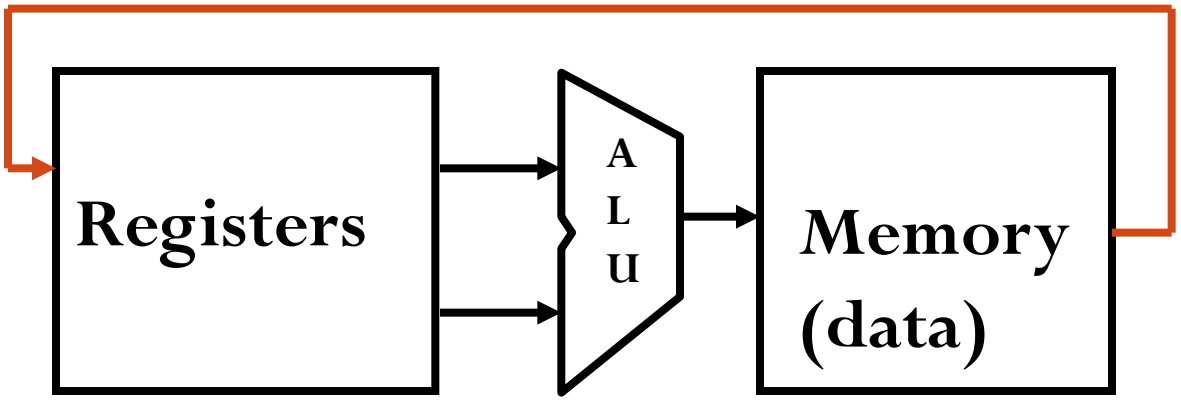


≤ 5 Steps in Executing MIPS Subset

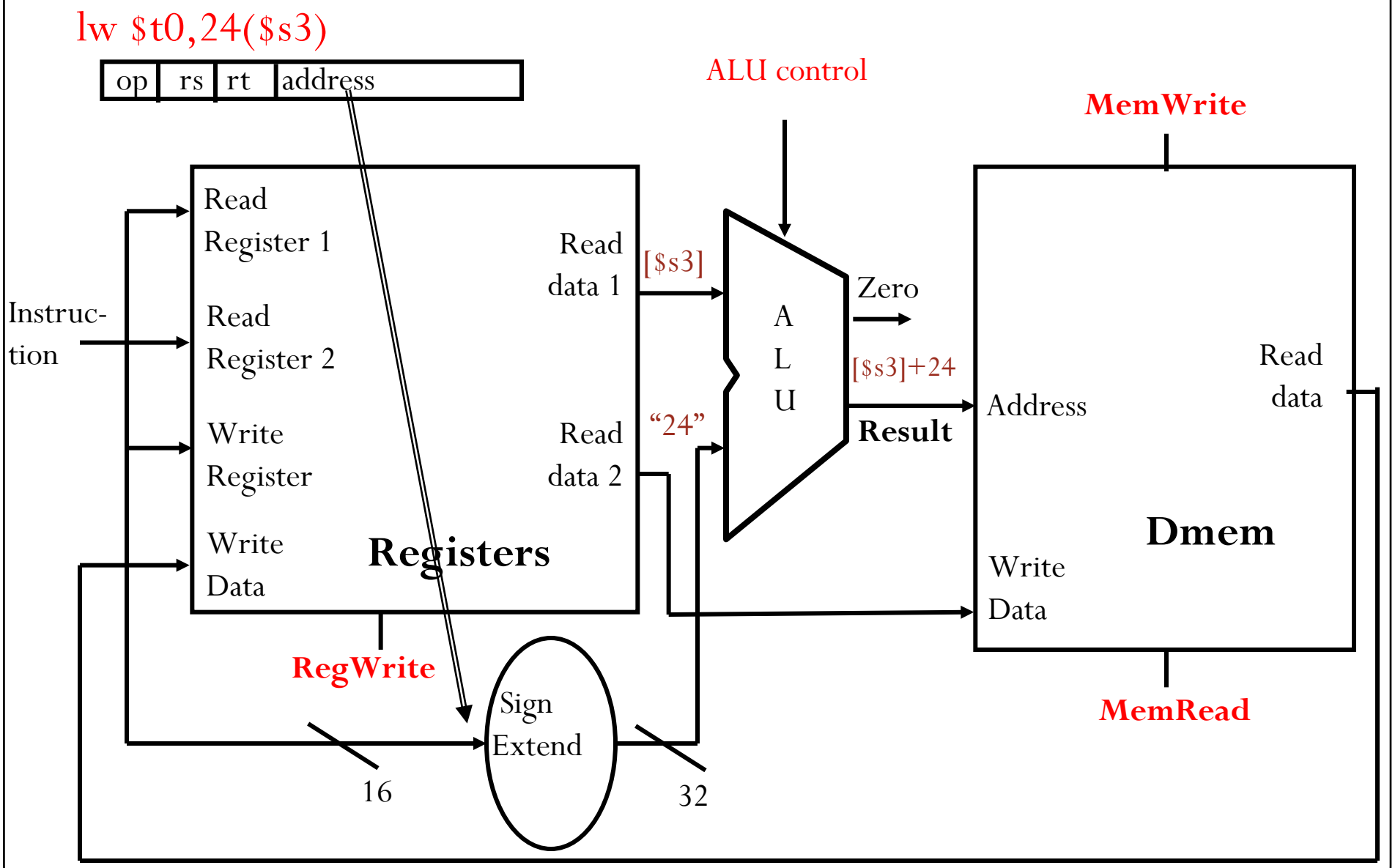
° **5th step only for lw; sw does not have this step**

EX: for lw: Write Result

$$Reg[rt] \Leftarrow Data$$



Datapath Step 3-5: Load/Store



Datapath Step 3-5 : R-form + Load/Store